

LIBRARY MANAGEMENT SYSTEM

[SCREENSHOTS]

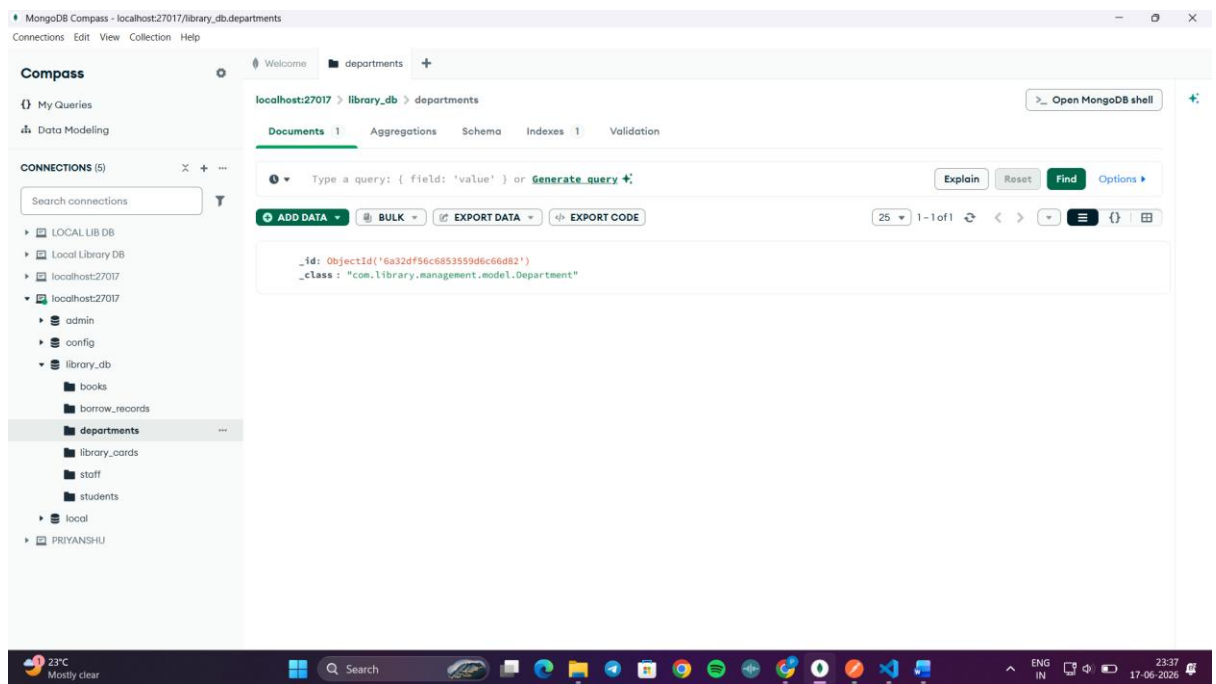
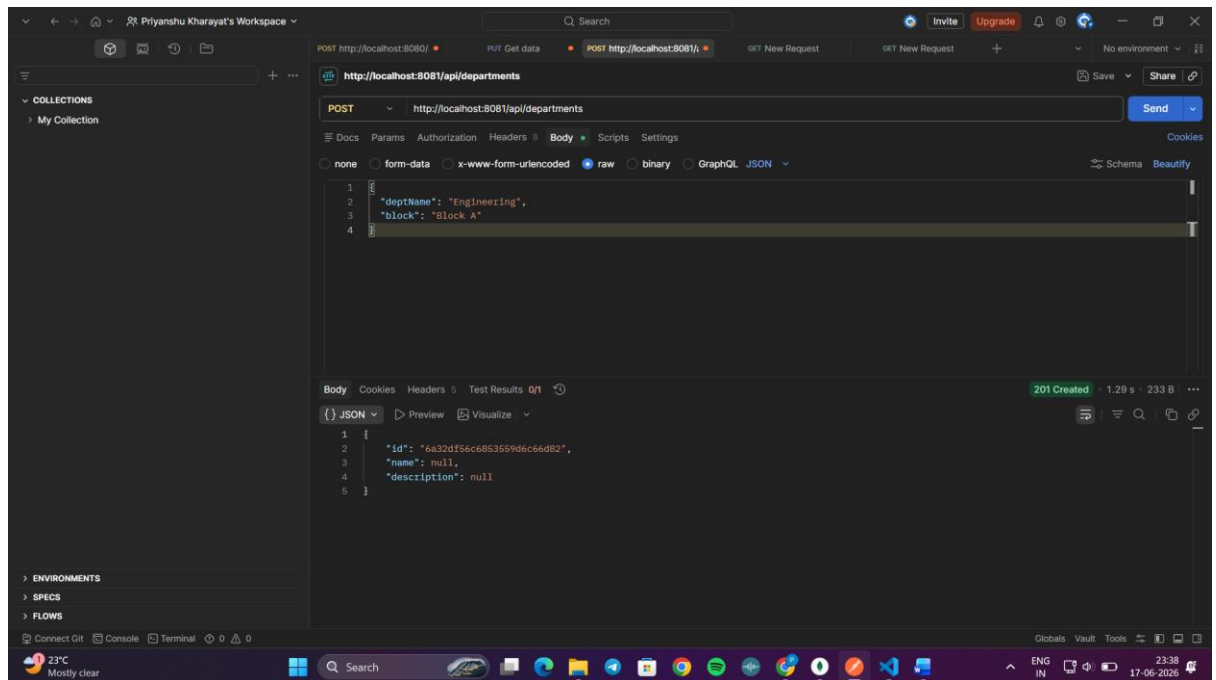
SPRING BOOT RUN

```
File Edit Selection View Go Run Terminal Help LIBRARY MANAGEMENT SYSTEM Sign In
CHAT
TERMINAL
PS C:\Users\priya\Music\LIBRARY MANAGEMENT SYSTEM> .\mvnw -q spring-boot:run
2026-06-17T23:28:31.483+05:30 INFO 27088 --- [library-management] [ main] c.l.m.LibraryManagementSystemApplication : Starting LibraryManagementSystemApplication using Java 25
.0.1 with PID 27088 (C:\Users\priya\Music\LIBRARY MANAGEMENT SYSTEM\target\classes started by priya in C:\Users\priya\Music\LIBRARY MANAGEMENT SYSTEM)
2026-06-17T23:28:31.492+05:30 INFO 27088 --- [library-management] [ main] c.l.m.LibraryManagementSystemApplication : No active profile set, falling back to 1 default profile:
"default"
2026-06-17T23:28:32.432+05:30 INFO 27088 --- [library-management] [ main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data MongoDB repositories in DEFAULT
mode.
2026-06-17T23:28:32.543+05:30 INFO 27088 --- [library-management] [ main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 106 ms. Found
6 MongoDB repository interfaces.
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.apache.tomcat.jni.Library in an unnamed module (file:/C:/Users/priya/.m2/repository/org/apache/tomcat/embed/tomcat-embed-core/10.1
.19/tomcat-embed-core-10.1.19.jar)
WARNING: Use --enable-native-access=ALL to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled
2026-06-17T23:28:33.065+05:30 INFO 27088 --- [library-management] [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8081 (http)
2026-06-17T23:28:33.088+05:30 INFO 27088 --- [library-management] [ main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-06-17T23:28:33.089+05:30 INFO 27088 --- [library-management] [ main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.19]
2026-06-17T23:28:33.171+05:30 INFO 27088 --- [library-management] [ main] o.a.c.c.g.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-06-17T23:28:33.172+05:30 INFO 27088 --- [library-management] [ main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1
592 ms
2026-06-17T23:28:33.422+05:30 INFO 27088 --- [library-management] [ main] org.mongodb.driver.client : MongoClient with metadata {"driver": {"name": "mongo-java
-driver|sync|spring-boot", "version": "4.11.1"}, "os": {"type": "Windows", "name": "Windows 11", "architecture": "amd64", "version": "10.0"}, "platform": "Java/Oracle Corporation/25.0.1
48-ITS-27"} created with settings MongoClientSettings(readPreference=primary, writeConcern=WriteConcern(w=null, wtimeout=null ms, journal=null), retryWrites=true, retryReads=true, readC
oncern=ReadConcern(level=null), credential=null, transportSettings=null, streamFactoryFactory=null, commandListeners=[], codeRegistry=ProvidersCodeRegistry(codeProviders=[ValueCodeP
rovider(), BsonValueCodeProvider(), DBRefCodeProvider(), DBObjectCodeProvider(), DocumentCodeProvider(), CollectionCodeProvider(), IterableCodeProvider(), MapCodeProvider(), GeoJ
sonCodeProvider(), GridFSFileCodeProvider(), JSR310CodeProvider(), JsonObjectCodeProvider(), BsonCodeProvider(), EnumCodeProvider(), com.mongodb.client.model.mql.ExpressionCodePr
ovider@63c3d2cd, com.mongodb.jdbc.jdbc.Driver@557a84fe, com.mongodb.kotlin.driver@6ddee370]), loggerSettings=LoggerSettings(maxDocumentLength=1000), clusterSettings={host
s=[localhost:27017], srvServiceName=mongodb, mode=SINGLE, requiredClusterType=UNIQUM, requiredReplicasetName=null, serverSelector=null, clusterListeners=[]}, serverSelectionTimeou
t=30000 ms, localThreshold=15 ms}, socketSettings=SocketSettings(connectTimeoutMS=10000, readTimeoutMS=0, receiveBufferSize=0, proxySettings=ProxySettings(host=null, port=null, use
rname=null, password=null)), heartbeatSocketSettings=SocketSettings(connectTimeoutMS=10000, readTimeoutMS=10000, receiveBufferSize=0, proxySettings=ProxySettings(host=null, port=null, us
ername=null, password=null)), connectionPoolSettings=ConnectionPoolSettings(maxSize=100, minSize=0, maxWaitTimeMS=120000, maxConnectionLifeTimeMS=0, maxConnectionIdleTimeMS=0, mainten
ceInitialDelayMS=0, maintenanceFrequencyMS=60000, connectionPoolListeners=[], maxConnecting=2), serverSettings=ServerSettings(heartbeatFrequencyMS=10000, minHeartbeatFrequencyMS=500, se
rverListeners=[], serverMonitorListeners=[]), sslSettings=SSLSettings(enabled=false, invalidHostNameAllowed=false, context=null), applicationName=null, compressorList=[], uuidRepr
esentation=JAVA_LEGACY, serverApi=null, autoEncryptionSettings=null, dnsClient=null, inetAddressResolver=null, contextProvider=null)
2026-06-17T23:28:33.467+05:30 INFO 27088 --- [library-management] [localhost:27017] org.mongodb.driver.cluster : Monitor thread successfully connected to server with desc
ription ServerDescription(address=localhost:27017, type=STANDALONE, state=CONNECTED, ok=true, minWireVersion=0, maxWireVersion=28, maxDocumentSize=16777216, logicalSessionTimeoutMinutes
CHAT
0 1 Java: Ready
23°C Mostly clear
```

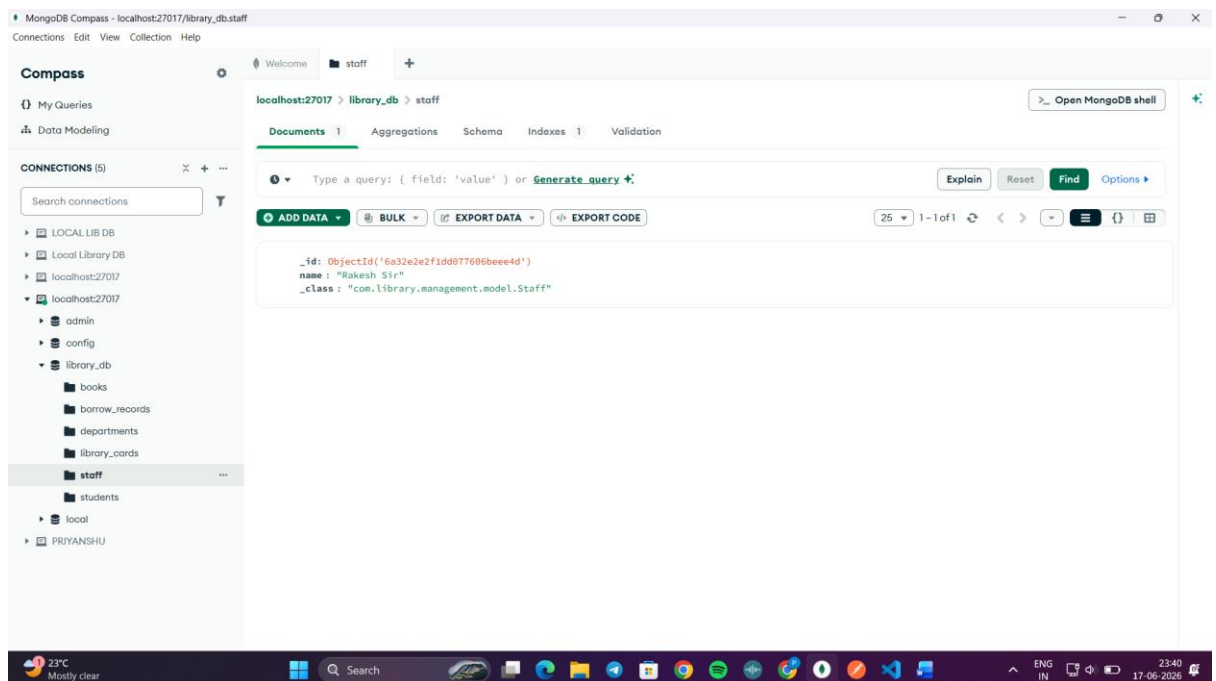
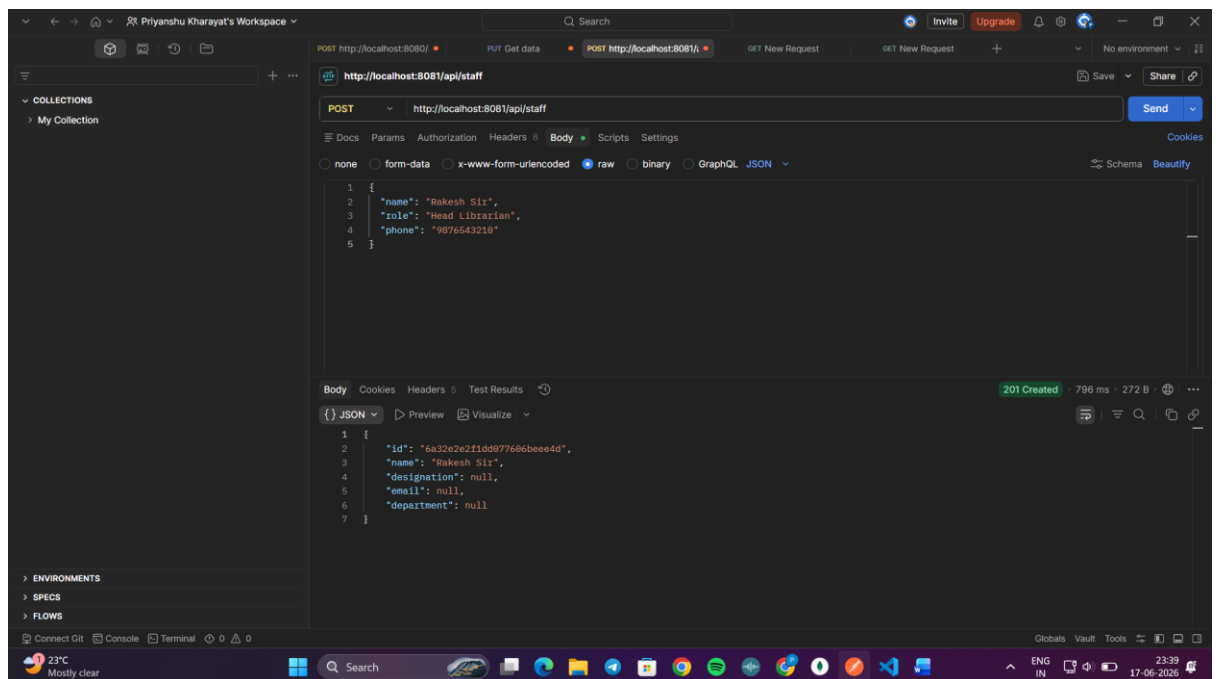
```
2026-06-17T23:28:33.422+05:30 INFO 27088 --- [library-management] [ main] org.mongodb.driver.client : MongoClient with metadata {"driver": {"name": "mongo-java
-driver|sync|spring-boot", "version": "4.11.1"}, "os": {"type": "Windows", "name": "Windows 11", "architecture": "amd64", "version": "10.0"}, "platform": "Java/Oracle Corporation/25.0.1
48-ITS-27"} created with settings MongoClientSettings(readPreference=primary, writeConcern=WriteConcern(w=null, wtimeout=null ms, journal=null), retryWrites=true, retryReads=true, readC
oncern=ReadConcern(level=null), credential=null, transportSettings=null, streamFactoryFactory=null, commandListeners=[], codeRegistry=ProvidersCodeRegistry(codeProviders=[ValueCodeP
rovider(), BsonValueCodeProvider(), DBRefCodeProvider(), DBObjectCodeProvider(), DocumentCodeProvider(), CollectionCodeProvider(), IterableCodeProvider(), MapCodeProvider(), GeoJ
sonCodeProvider(), GridFSFileCodeProvider(), JSR310CodeProvider(), JsonObjectCodeProvider(), BsonCodeProvider(), EnumCodeProvider(), com.mongodb.client.model.mql.ExpressionCodePr
ovider@63c3d2cd, com.mongodb.jdbc.jdbc.Driver@557a84fe, com.mongodb.kotlin.driver@6ddee370]), loggerSettings=LoggerSettings(maxDocumentLength=1000), clusterSettings={host
s=[localhost:27017], srvServiceName=mongodb, mode=SINGLE, requiredClusterType=UNIQUM, requiredReplicasetName=null, serverSelector=null, clusterListeners=[]}, serverSelectionTimeou
t=30000 ms, localThreshold=15 ms}, socketSettings=SocketSettings(connectTimeoutMS=10000, readTimeoutMS=0, receiveBufferSize=0, proxySettings=ProxySettings(host=null, port=null, use
rname=null, password=null)), heartbeatSocketSettings=SocketSettings(connectTimeoutMS=10000, readTimeoutMS=10000, receiveBufferSize=0, proxySettings=ProxySettings(host=null, port=null, us
ername=null, password=null)), connectionPoolSettings=ConnectionPoolSettings(maxSize=100, minSize=0, maxWaitTimeMS=120000, maxConnectionLifeTimeMS=0, maxConnectionIdleTimeMS=0, mainten
ceInitialDelayMS=0, maintenanceFrequencyMS=60000, connectionPoolListeners=[], maxConnecting=2), serverSettings=ServerSettings(heartbeatFrequencyMS=10000, minHeartbeatFrequencyMS=500, se
rverListeners=[], serverMonitorListeners=[]), sslSettings=SSLSettings(enabled=false, invalidHostNameAllowed=false, context=null), applicationName=null, compressorList=[], uuidRepr
esentation=JAVA_LEGACY, serverApi=null, autoEncryptionSettings=null, dnsClient=null, inetAddressResolver=null, contextProvider=null)
2026-06-17T23:28:33.467+05:30 INFO 27088 --- [library-management] [localhost:27017] org.mongodb.driver.cluster : Monitor thread successfully connected to server with desc
ription ServerDescription(address=localhost:27017, type=STANDALONE, state=CONNECTED, ok=true, minWireVersion=0, maxWireVersion=28, maxDocumentSize=16777216, logicalSessionTimeoutMinutes
30, roundTripTimeMS=332033000)
2026-06-17T23:28:34.369+05:30 INFO 27088 --- [library-management] [ main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8081 (http) with context path ''
2026-06-17T23:28:34.380+05:30 INFO 27088 --- [library-management] [ main] c.l.m.LibraryManagementSystemApplication : Started LibraryManagementSystemApplication in 3.596 secon
ds (process running for 4.312)
CHAT
1 Java: Ready
23°C Mostly clear
```

#POST (Creating Data)

1. Create a Department



2. Add a Staff Member



3. Register a Student

The image shows two screenshots from a Windows desktop. The top screenshot is from a REST client (Postman) showing a successful POST request to `http://localhost:8081/api/students`. The request body is a JSON object: `{ "name": "Aditya Sharma", "course": "BCA", "rollNo": "45" }`. The response status is `201 Created` with a response time of `49 ms` and a content size of `270 B`. The response body is a JSON object: `{ "id": "6a32e3f3f1dd977606bee4e", "name": "Aditya Sharma", "rollNo": "45", "email": null, "department": null }`.

The bottom screenshot is from MongoDB Compass, showing the `students` collection in the `library_db` database. The `Documents` tab is selected, showing a single document with the following fields: `_id` (ObjectId), `name` (Aditya Sharma), `rollNo` (45), and `_class` (com.library.management.model.Student).

4. Issue a Library Card

The image shows two screenshots from a Windows desktop. The top screenshot is from a REST client (Postman) showing a successful POST request to `http://localhost:8081/api/library-cards`. The request body is a JSON object with the following data:

```
{
  "studentId": "6a32e3f3f1dd977686beee4e",
  "status": "Active",
  "validupto": "2827-05-01"
}
```

The response is a 201 Created status with a response time of 75 ms and a body size of 387 B. The response body is a JSON object:

```
{
  "id": "6a32e58af1dd977686beee4f",
  "cardNumber": null,
  "student": {
    "id": "6a32e3f3f1dd977686beee4e",
    "name": "Aditya Sharma",
    "rollNo": "45",
    "email": null,
    "department": null
  },
  "issueDate": null,
  "status": "Active"
}
```

The bottom screenshot is from MongoDB Compass, showing the `library_cards` collection in the `library_db` database. The document shown in the `Documents` tab is:

```
{
  "_id": ObjectId('6a32e58af1dd977686beee4f'),
  "student": DBRef('students', '6a32e3f3f1dd977686beee4e'),
  "status": "Active",
  "_class": "com.library.management.model.LibraryCard"
}
```

The left sidebar of MongoDB Compass shows the database structure, including collections like `books`, `borrow_records`, `departments`, `library_cards`, `staff`, and `students`.

5. Add a New Book

The image shows two screenshots from a Windows desktop. The top screenshot is from a REST client (Postman) showing a successful POST request to `http://localhost:8081/api/books`. The request body is a JSON object representing a book. The response status is `201 Created` with a response time of `65 ms` and a body size of `304 B`. The response body is a JSON object with an `_id`, `author`, `title`, `isbn`, `totalCopies`, `availableCopies`, and `_class`.

```
POST http://localhost:8081/api/books
```

```
{
  "bookName": "Let Us C",
  "author": "Yashavant Kanetkar",
  "category": "Computer Science",
  "isbn": "9788183331630",
  "totalCopies": 5,
  "status": "Available"
}
```

The bottom screenshot is from MongoDB Compass, showing the `books` collection in the `library_db` database. The document is displayed in the `Documents` tab, showing the same data as the REST client response.

```
{
  "_id": "6a32e5fffd077606bee50",
  "author": "Yashavant Kanetkar",
  "title": null,
  "isbn": "9788183331630",
  "totalCopies": 5,
  "availableCopies": 0,
  "_class": "com.library.management.model.Book"
}
```

6. Borrow a Book (Borrow Record)

The image shows a REST client interface (Postman) and MongoDB Compass. The REST client is configured with a POST request to `http://localhost:8081/api/borrow-records`. The request body is a JSON object:

```
1 {
2   "studentId": "6a32e3f3f1dd077606beee4e",
3   "bookId": "6a32e5ffff1dd077606beee50",
4   "issuedBy": "Rakesh Sir",
5   "dueDate": "2026-06-25"
6 }
```

The response status is **201 Created** with a response time of 147 ms and a body size of 525 B. The response body is a JSON object:

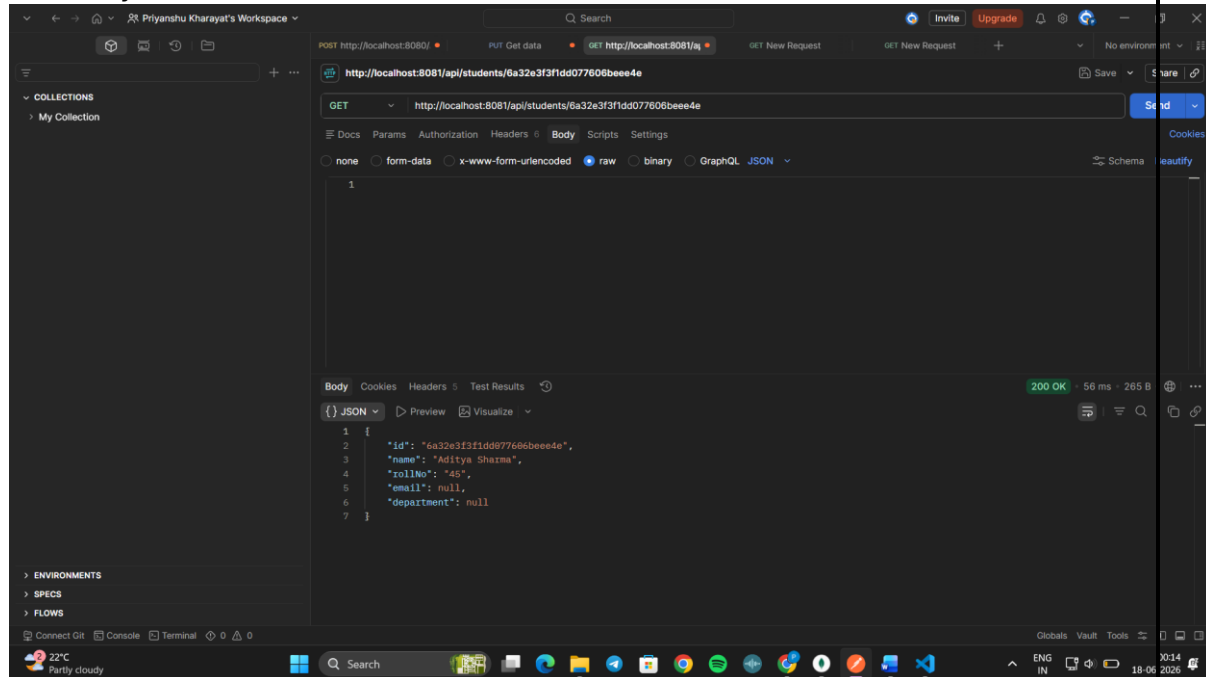
```
1 {
2   "id": "6a32e939a453c779a23d0aec",
3   "student": {
4     "id": "6a32e3f3f1dd077606beee4e",
5     "name": "Aditya Sharma",
6     "rollNo": "45",
7     "email": null,
8     "department": null
9   },
10  "book": {
11    "id": "6a32e5ffff1dd077606beee50",
12    "title": null,
13    "author": "Yashavant Kanetkar",
14    "isbn": "9788183331630",
15  }
16 }
```

MongoDB Compass shows the `borrow_records` collection in the `library_db` database. The query results show the following document:

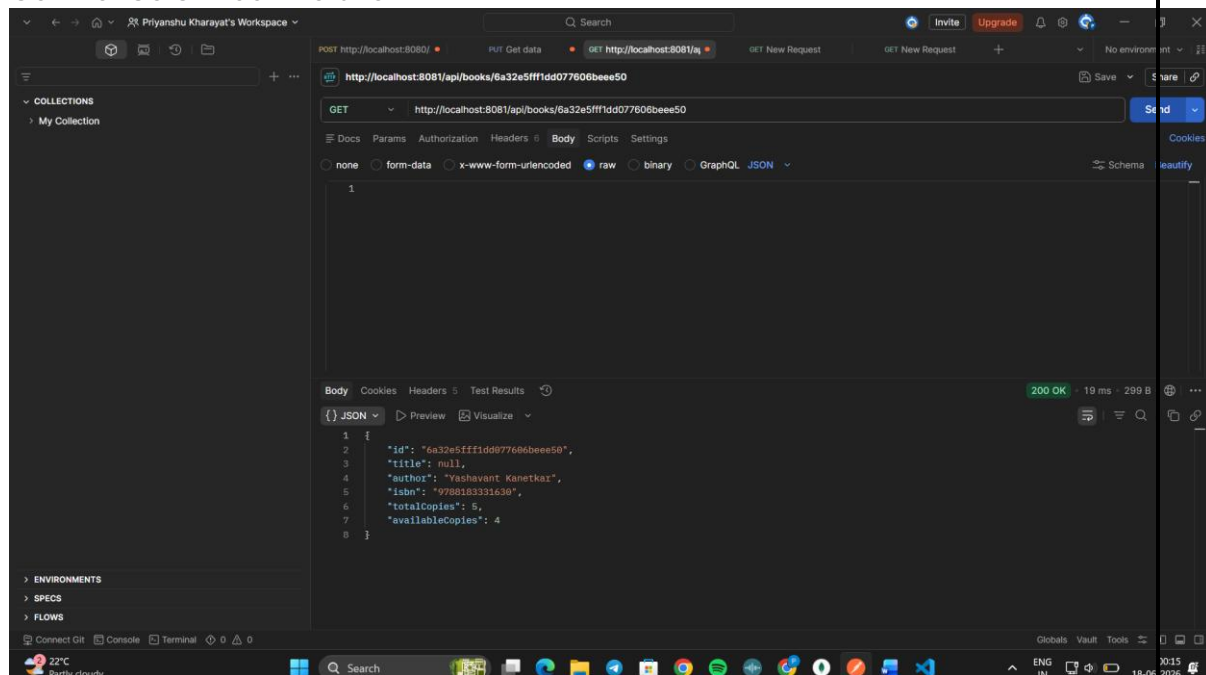
```
_id: ObjectId('6a32e939a453c779a23d0aec')
student: DBRef('students', '6a32e3f3f1dd077606beee4e')
book: DBRef('books', '6a32e5ffff1dd077606beee50')
issueDate: 2026-06-17T18:30:00.000+00:00
dueDate: 2026-07-01T18:30:00.000+00:00
status: "BORROWED"
_class: "com.library.management.model.BorrowRecord"
```


#GET (Retrieve Exact Records)

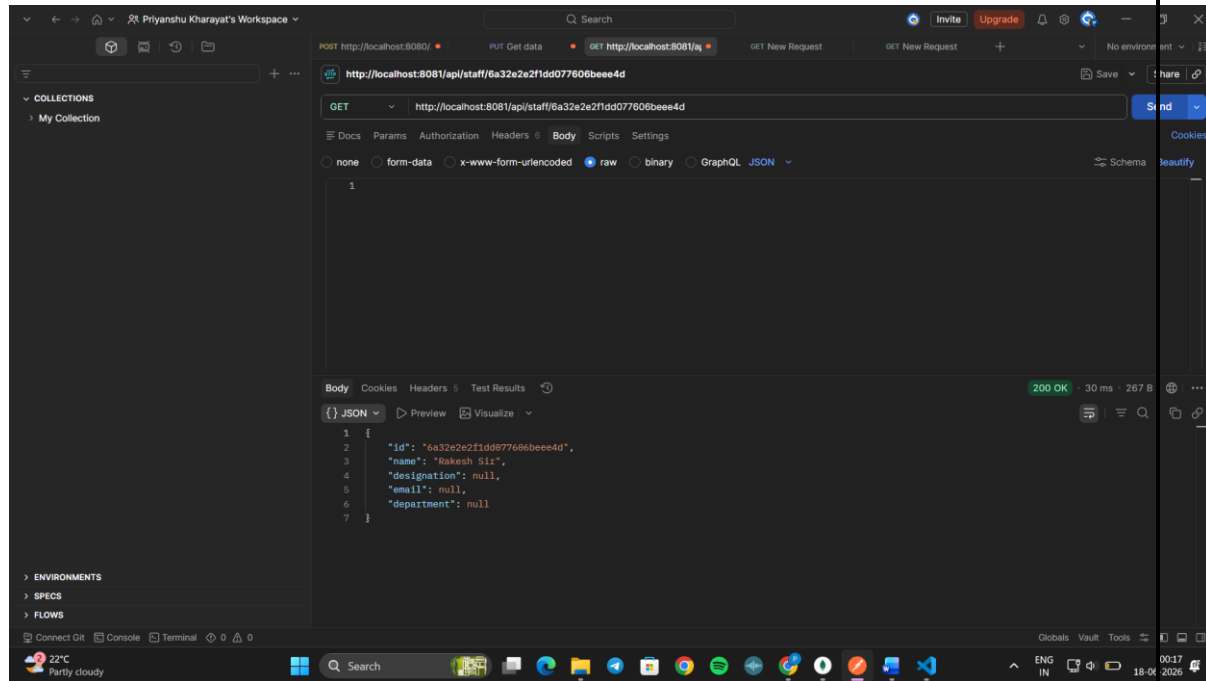
1. Get Aditya's Student Record



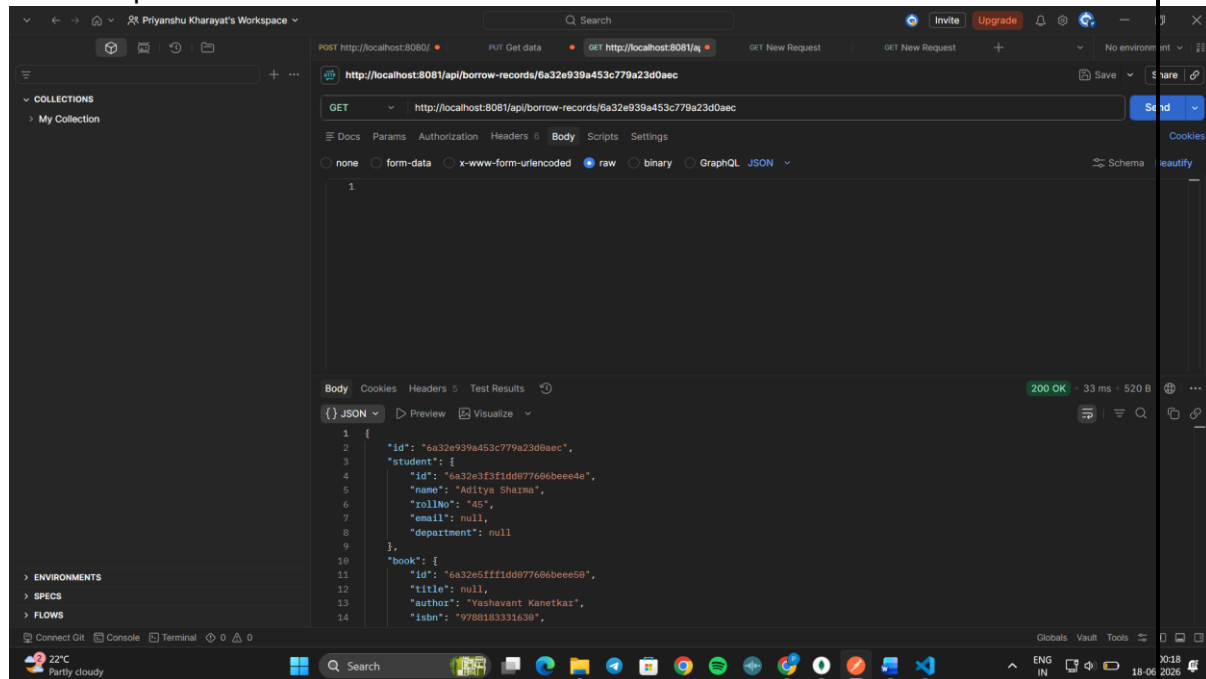
2. Get "Let Us C" Book Details



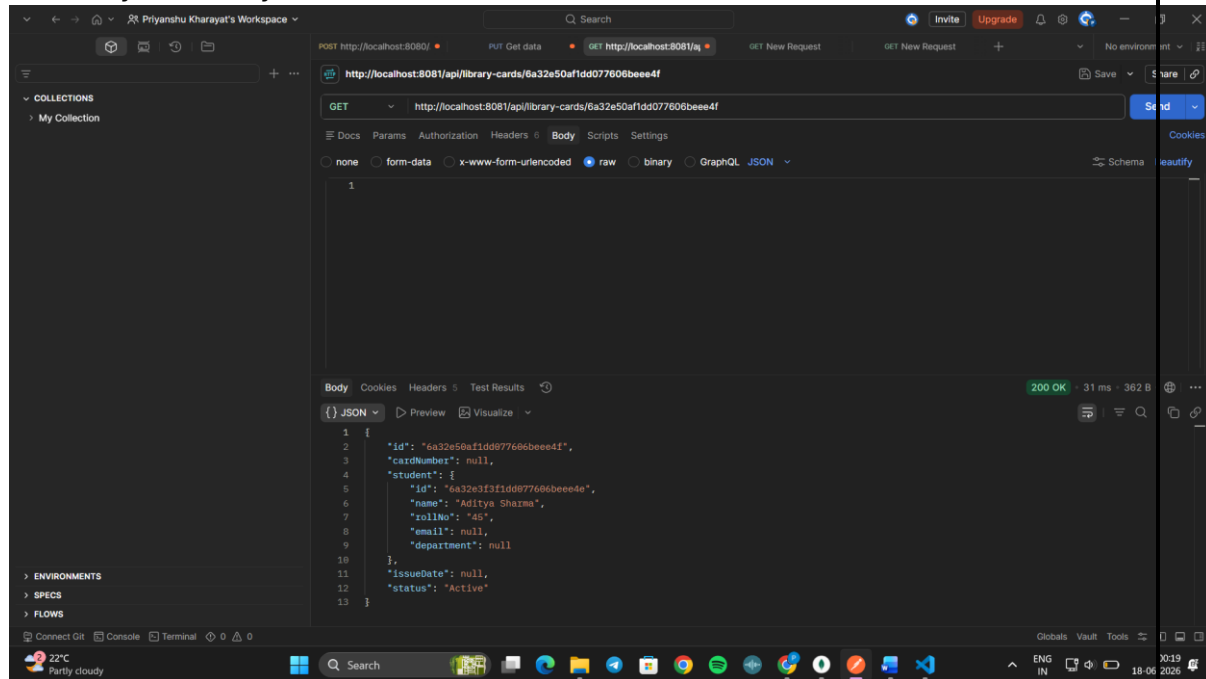
3. Get Rakesh sir Staff Profile



4. Get the specific Borrow Record

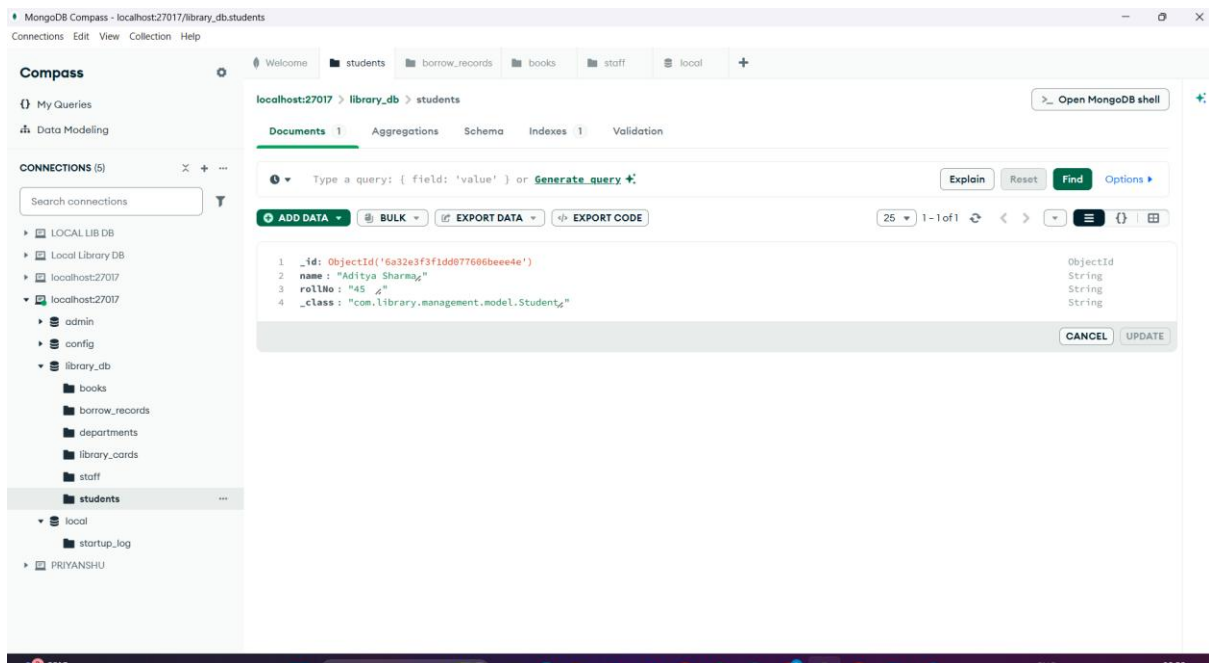
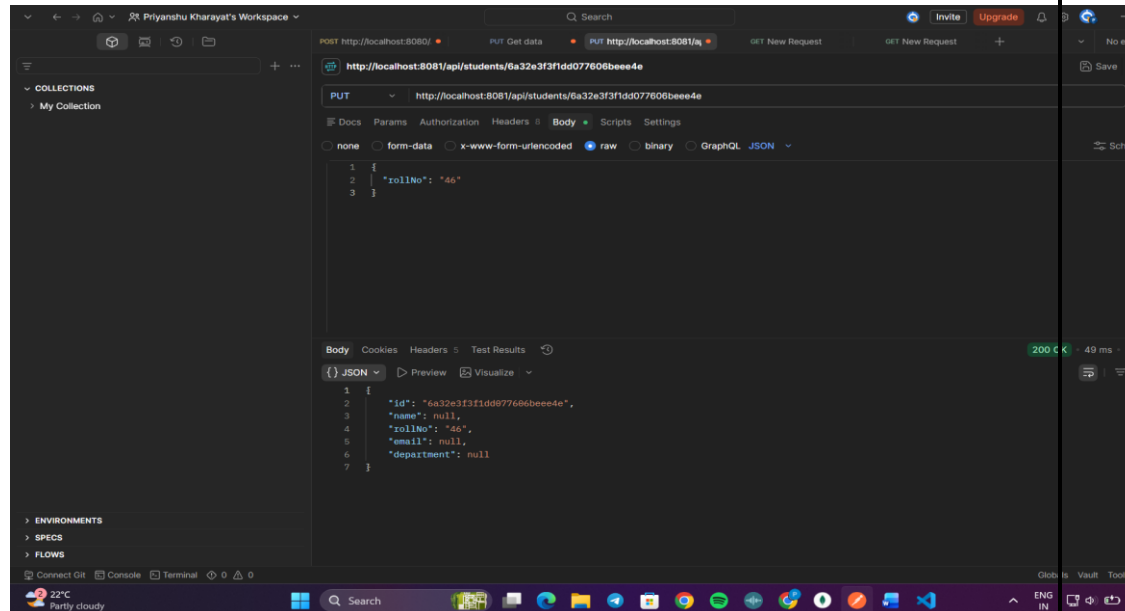


5. Get Aditya's Library Card



#PUT (Update Existing Records)

1. Update Aditya's Roll Number



2. Return the Borrowed Book (Update Borrow Status)

The image displays two software interfaces used for testing and managing a database. The top interface is a REST client (Postman) showing a PUT request to the endpoint `http://localhost:8081/api/borrow-records/6a32e939a453c779a23d0aec`. The request body is a JSON object:

```
{  "status": "RETURNED"}
```

. The response is a 200 OK status with a response time of 1.60s and a body size of 509B. The response body is a JSON object:

```
{  "id": "6a32e939a453c779a23d0aec",  "student": {    "id": "6a32e3f3f1dd077606beee4e",    "name": null,    "rollNo": "46",    "email": null,    "department": null  },  "book": {    "id": "6a32e5fff1dd077606beee50",    "title": null,    "author": "Yashavant Kanetkar",    "isbn": "9780183331630"  }}
```

The bottom interface is MongoDB Compass, showing the `library_db` database and the `borrow_records` collection. The document being updated is displayed in the Documents tab:

```
{  "_id": ObjectId('6a32e939a453c779a23d0aec'),  "student": DBRef('students', '6a32e3f3f1dd077606beee4e'),  "book": DBRef('books', '6a32e5fff1dd077606beee50'),  "issueDate": 2026-06-17T18:38:00.000+00:00,  "dueDate": 2026-07-01T18:38:00.000+00:00,  "status": "BORROWED",  "_class": "com.library.management.model.BorrowRecord"}
```

3. Decrease Available Book Copies

The image shows a REST client interface (likely Postman) and MongoDB Compass. The REST client is configured with a PUT request to `http://localhost:8081/api/books/6a32e5fff1dd077606beee50`. The request body is a JSON object:

```
{  "status": "RETURNED"}
```

. The response status is `200 OK` with a response body:

```
{  "id": "6a32e5fff1dd077606beee50",  "title": null,  "author": null,  "isbn": null,  "totalCopies": 0,  "availableCopies": 0}
```

. Below the REST client, MongoDB Compass is open, showing the `books` collection in the `library_db` database. The document with `_id: ObjectID('6a32e5fff1dd077606beee50')` is displayed, showing `totalCopies: 5` and `availableCopies: 5`.

REST Client Request:

```
PUT http://localhost:8081/api/books/6a32e5fff1dd077606beee50
```

Request Body:

```
{  "status": "RETURNED"}
```

Response: 200 OK

Response Body:

```
{  "id": "6a32e5fff1dd077606beee50",  "title": null,  "author": null,  "isbn": null,  "totalCopies": 0,  "availableCopies": 0}
```

MongoDB Compass:

Database: `library_db`, Collection: `books`

Document:

```
{  "_id": ObjectID("6a32e5fff1dd077606beee50"),  "author": "Yashavant Kanetkar",  "isbn": "9788183316369",  "totalCopies": 5,  "availableCopies": 5,  "_class": "com.library.management.model.Book"}
```

Update a book

4. Update a book

The image shows a REST client interface (Postman) and MongoDB Compass. The REST client is configured to send a PUT request to `http://localhost:8081/api/books/6a32e5fff1dd077606beee50` with a JSON body representing a book update. The response is a 200 OK status with a JSON body containing the updated book details.

REST Client Request:

```
PUT http://localhost:8081/api/books/6a32e5fff1dd077606beee50
```

Request Body (JSON):

```
{  "title": "Let Us C",  "author": "Yashavant Kanetkar",  "isbn": "9788183331630",  "totalCopies": 10,  "availableCopies": 8}
```

Response (200 OK):

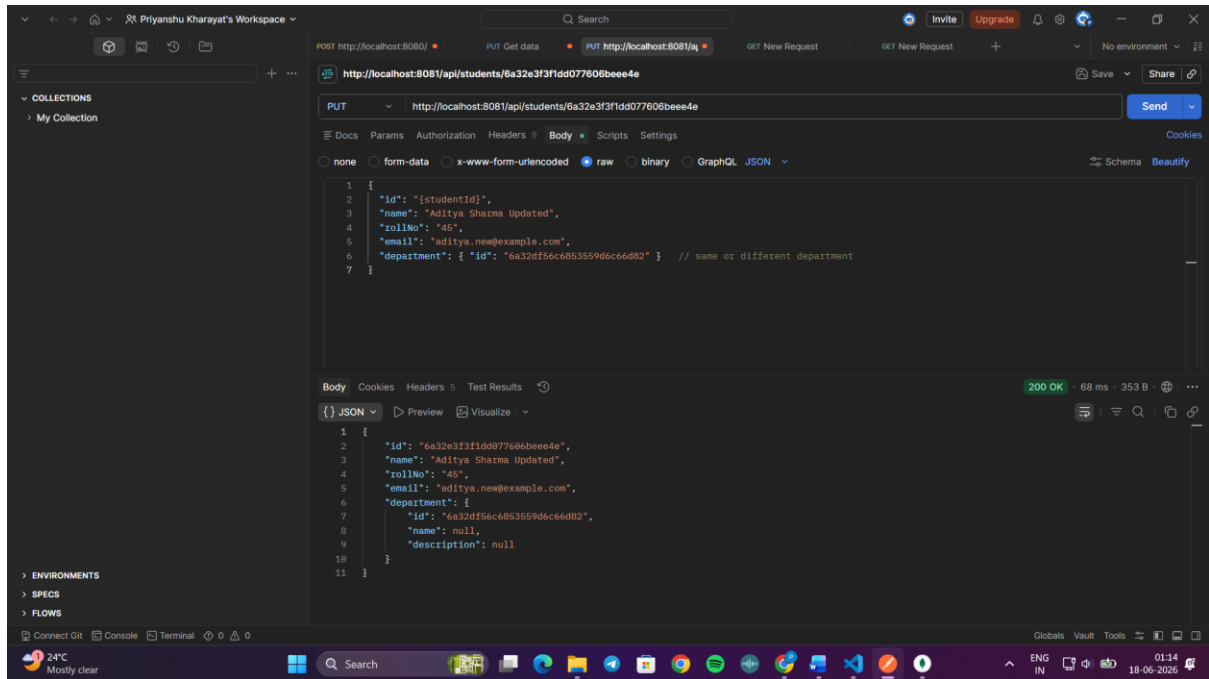
```
{  "id": "6a32e5fff1dd077606beee50",  "title": "Let Us C",  "author": "Yashavant Kanetkar",  "isbn": "9788183331630",  "totalCopies": 10,  "availableCopies": 8}
```

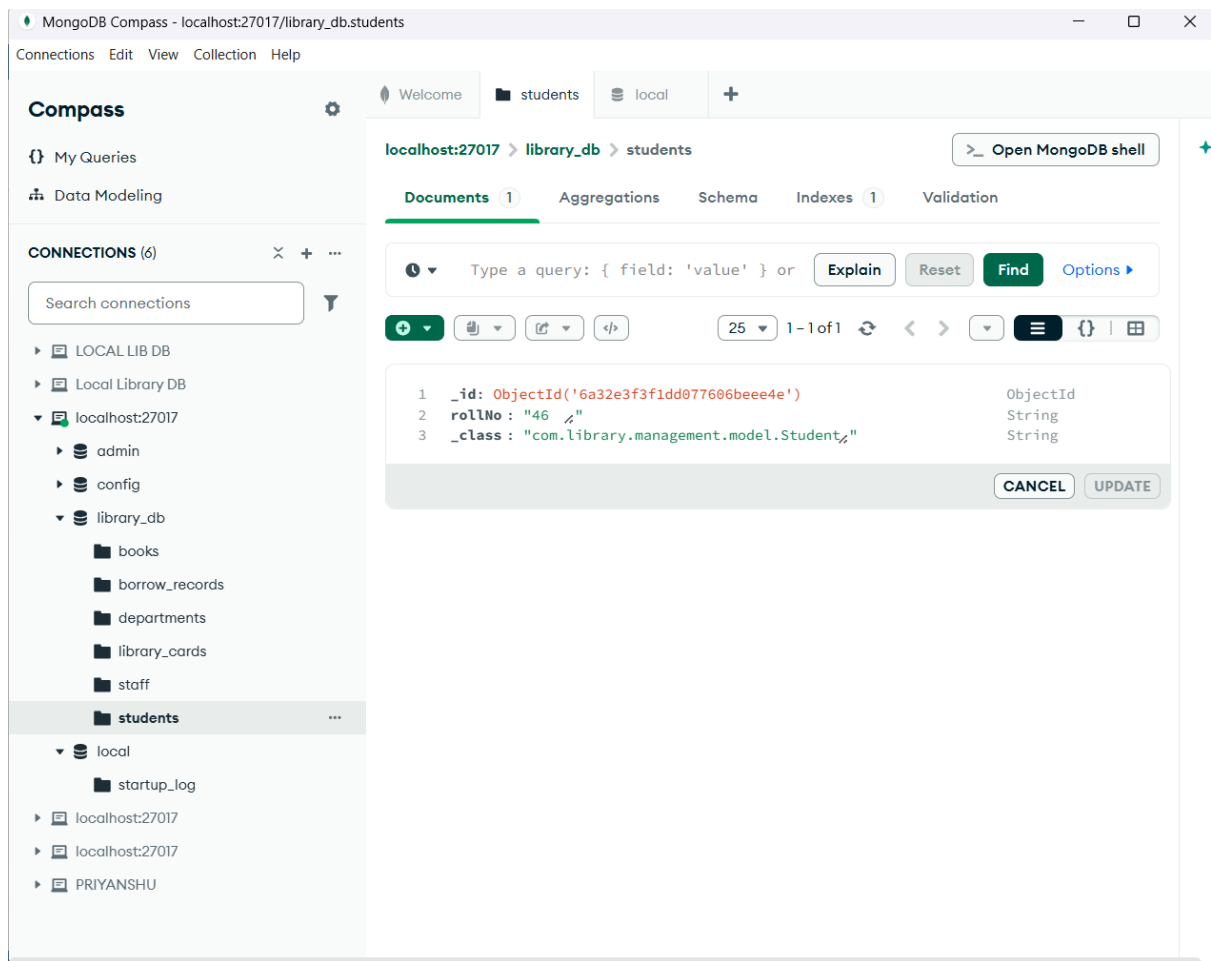
MongoDB Compass:

The MongoDB Compass interface shows the `books` collection in the `library_db` database. The document with `_id: ObjectId('6a32e5fff1dd077606beee50')` is displayed, showing the updated book details:

```
{  "_id": ObjectId('6a32e5fff1dd077606beee50'),  "title": "Let Us C",  "author": "Yashavant Kanetkar",  "isbn": "9788183331630",  "totalCopies": 10,  "availableCopies": 8,  "_class": "com.library.management.model.Book"}
```

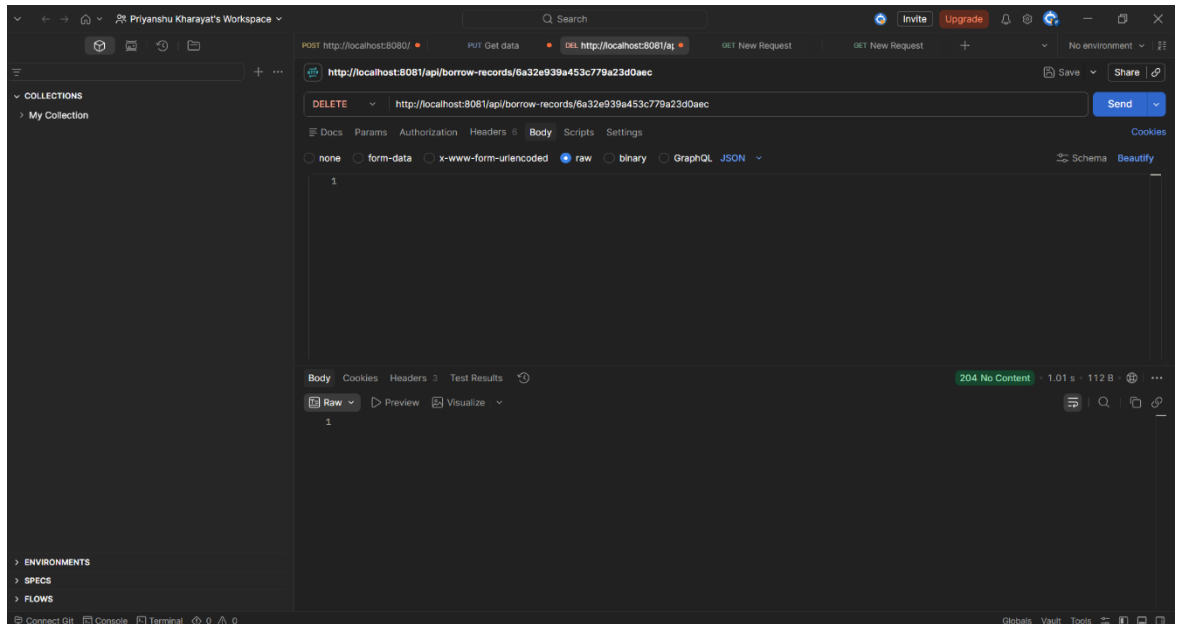
5. Change Department



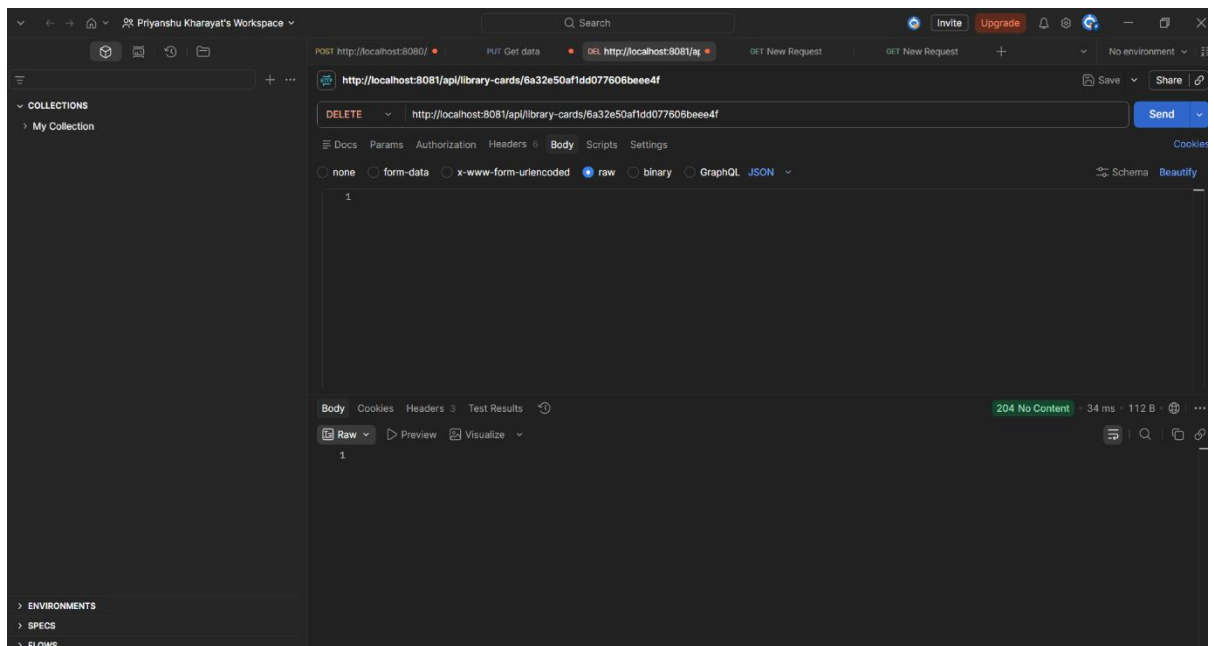


#DELETE

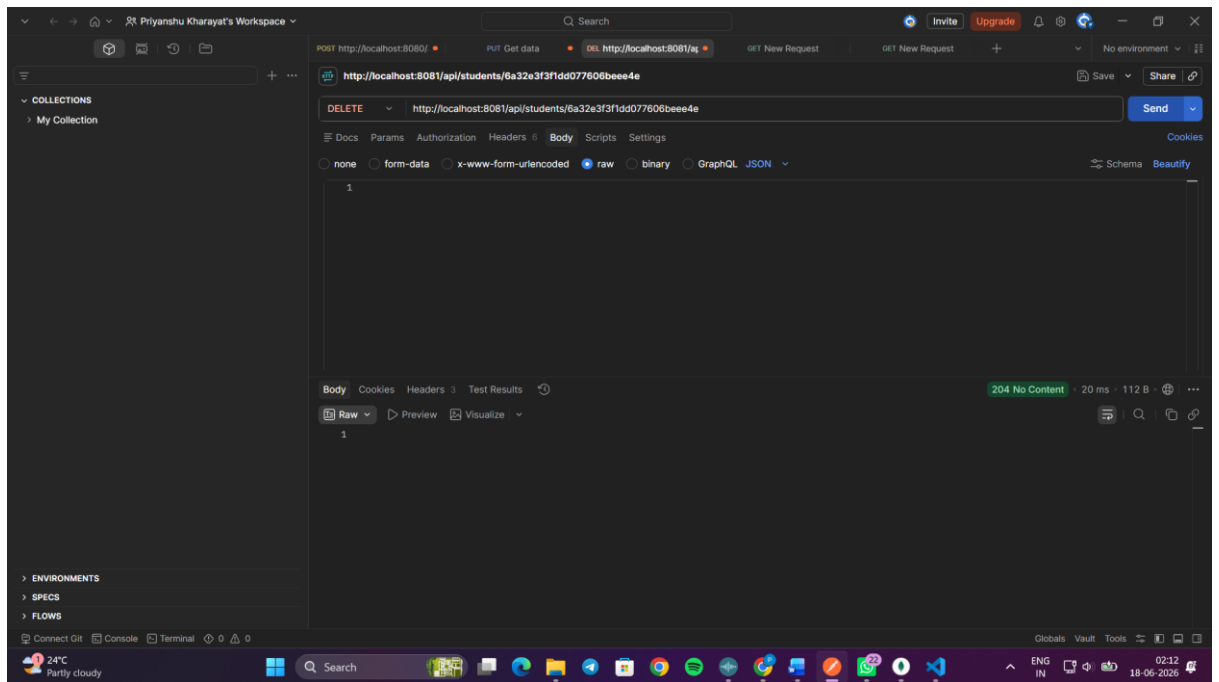
1. Delete borrow record



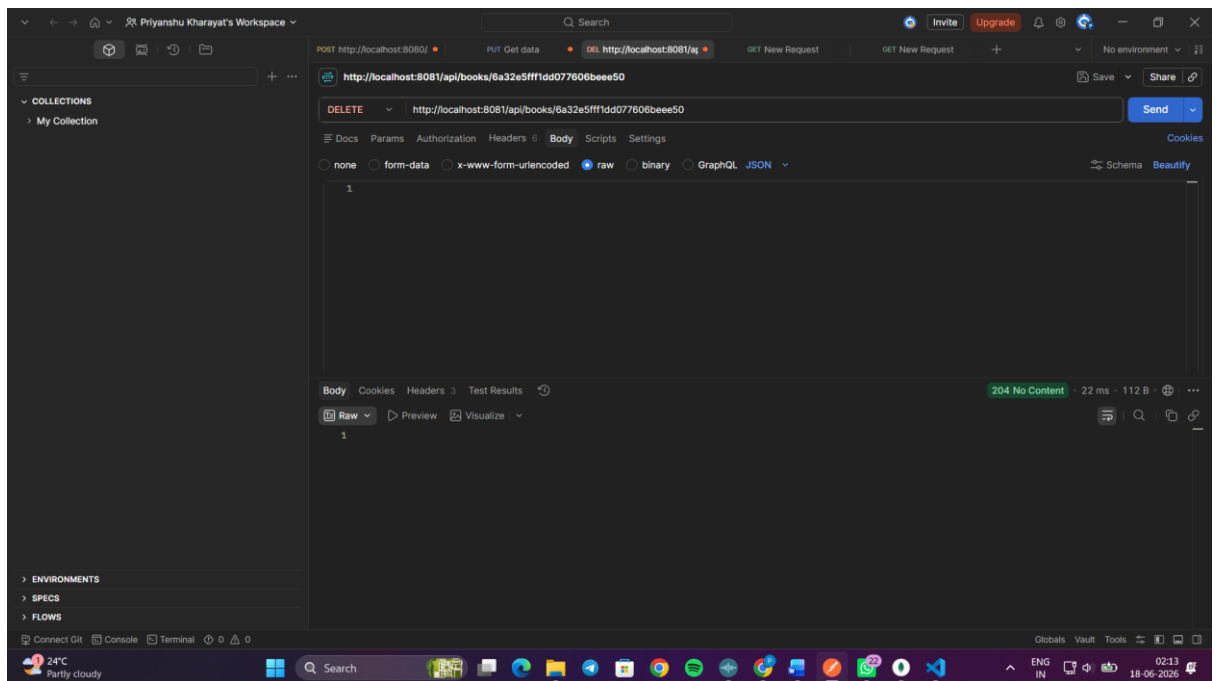
2. Delete Aditya's Library Card



3. Delete Aditya's Student Record

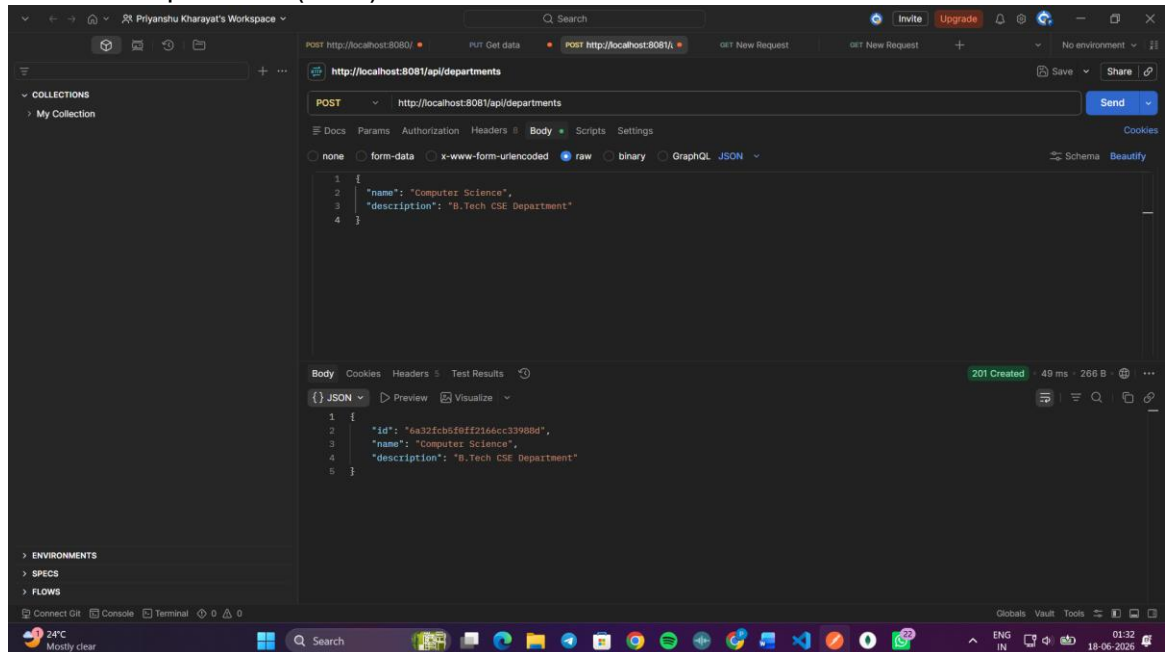


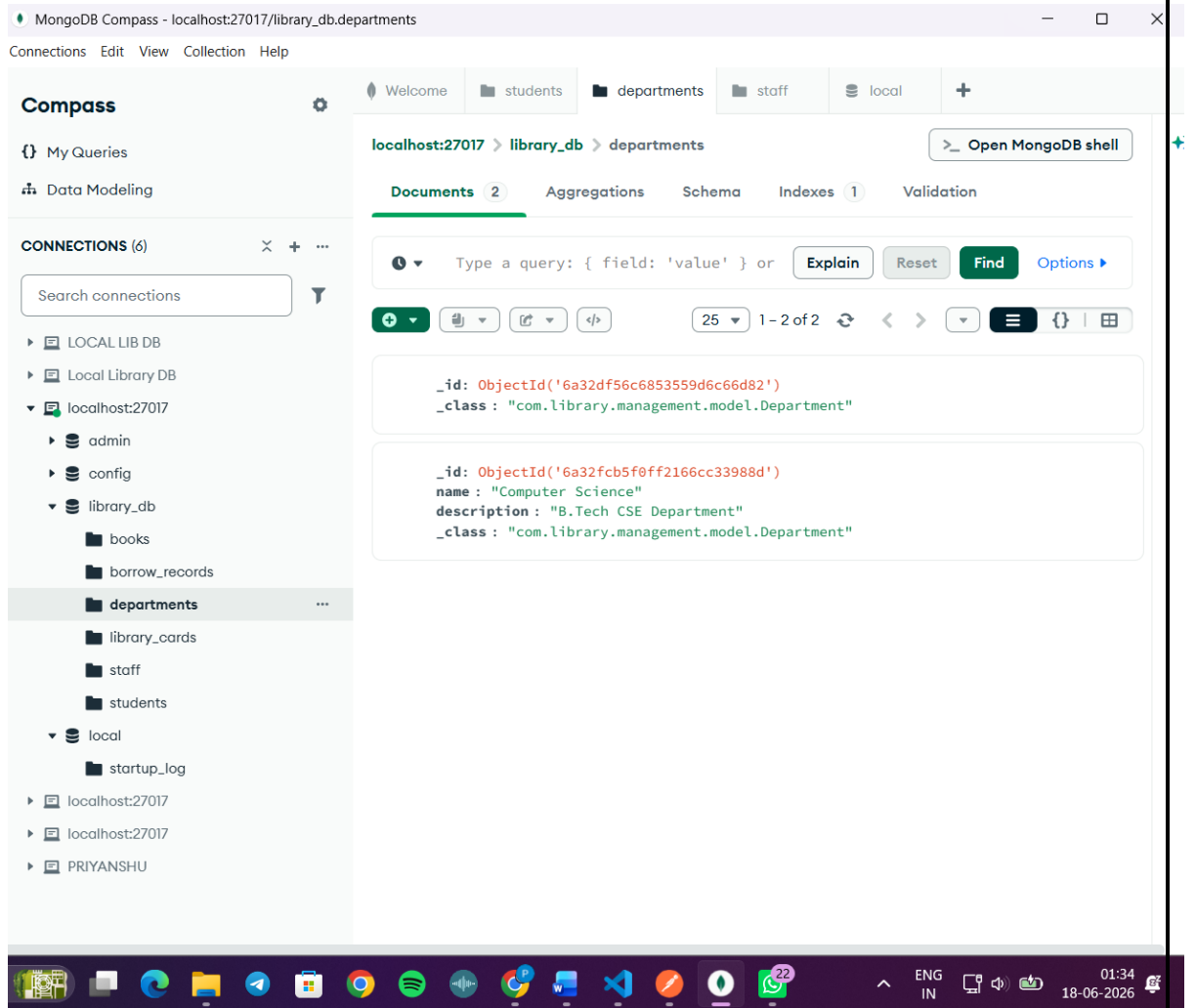
4. Delete the Book



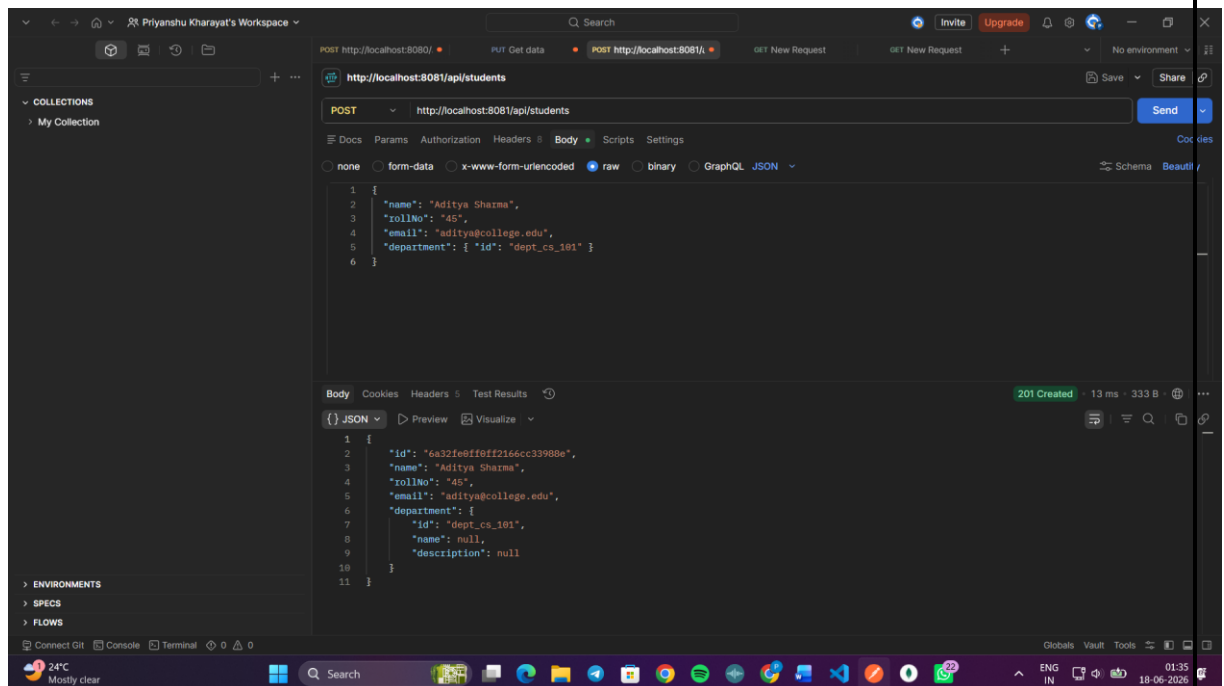
#MAPPINGS (ONE TO ONE, ONE TO MANY, MANY TO ONE, MANY TO MANY)

1. Create a department(Root)

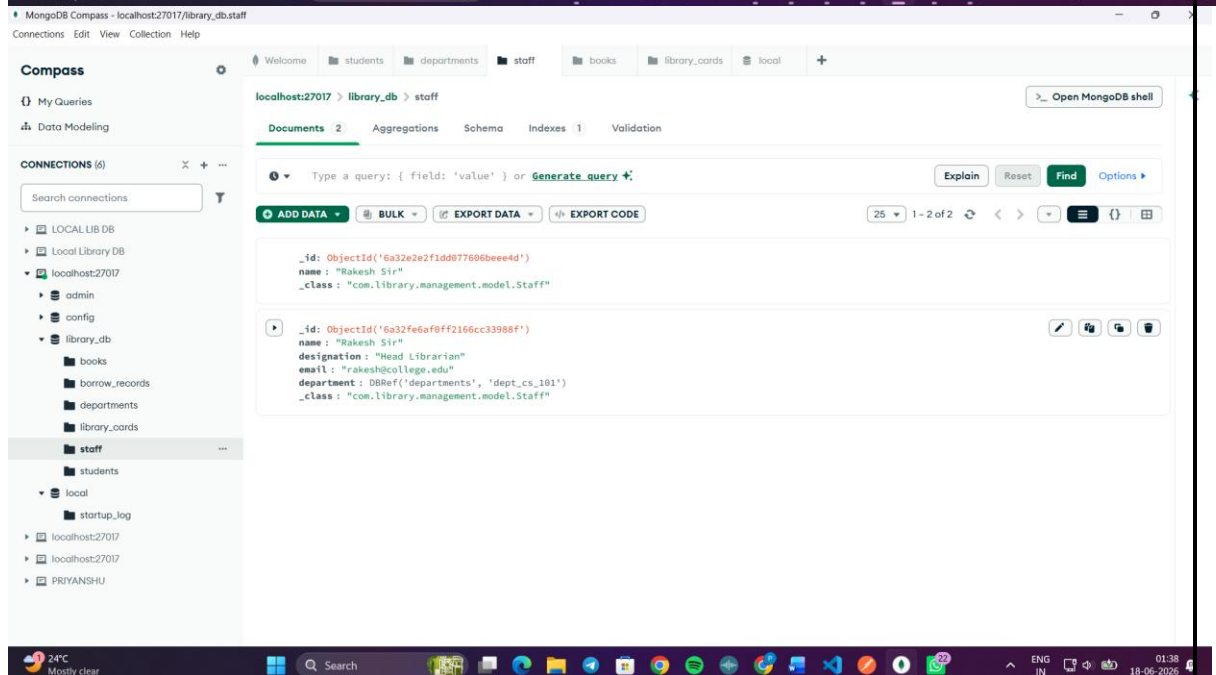
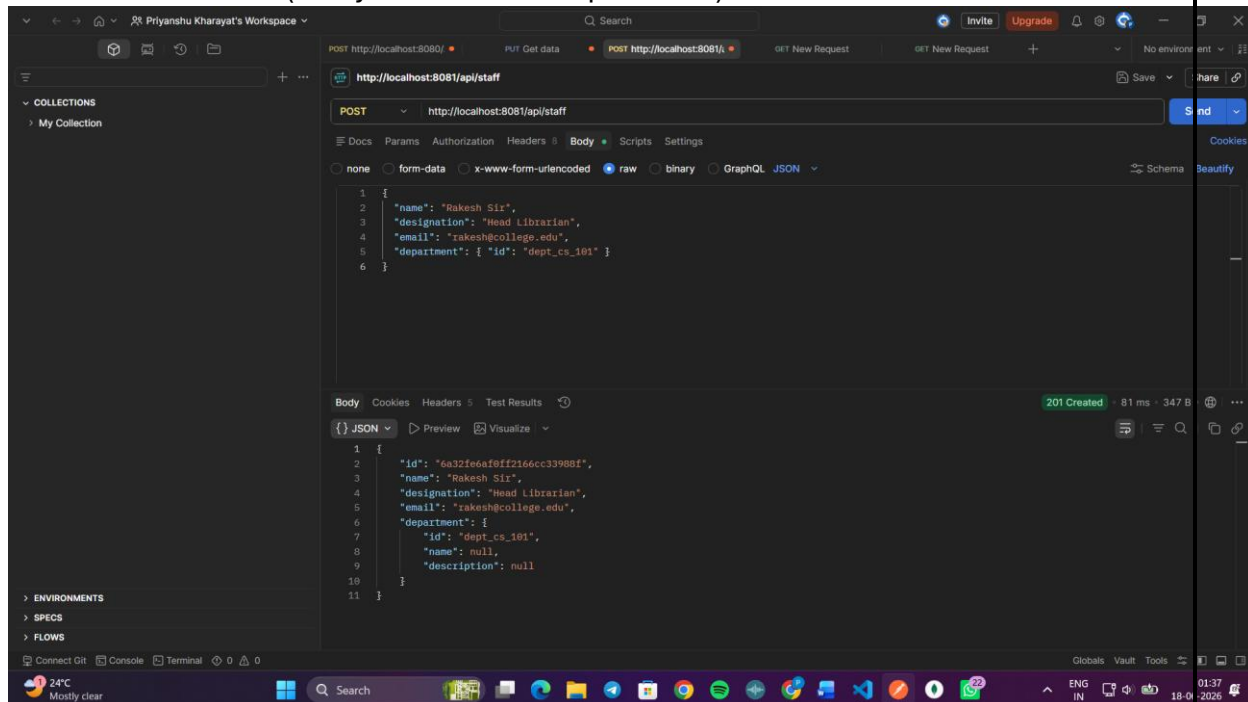




2. Register a Student (Many-to-One with Department)



3. Add a Staff Member (Many-to-One with Department)



4. Create a borrow record(Many to Many)

The image shows two screenshots illustrating the creation of a borrow record.

Top Screenshot (REST Client): A POST request is sent to `http://localhost:8081/api/borrow-records`. The request body is a JSON object:

```
1 {
2   "studentId": "6a32e3f3f1dd87768b6ee4e",
3   "bookId": "6a32e5fffd87768b6ee58",
4   "dueDate": "2026-07-01",
5   "issuedBy": "admin"
6 }
7
```

The response status is **201 Created**. The response body is a JSON object:

```
1 {
2   "id": "6a3381b9f0ff216cc33989e",
3   "student": {
4     "id": "6a32e3f3f1dd87768b6ee4e",
5     "name": "Aditya Sharma Updated",
6     "rollNo": "46",
7     "email": "aditya.new@example.com",
8     "department": {
9       "id": "6a32df56c685359dc66d02",
10      "name": null,
11      "description": null
12    }
13  },
14  "book": {
15    "id": "6a32e5fffd87768b6ee58",
16    "name": "The Great Gatsby",
17    "author": "F. Scott Fitzgerald",
18    "year": 1925,
19    "genre": "Fiction",
20    "status": "AVAILABLE"
21  },
22  "issueDate": "2026-06-17T18:30:00.000+00:00",
23  "dueDate": "2026-07-01T18:30:00.000+00:00",
24  "status": "RETURNED",
25  "_class": "com.library.management.model.BorrowRecord"
26 }
```

Bottom Screenshot (MongoDB Compass): The MongoDB Compass interface shows the `library_db` database and the `borrow_records` collection. The document is displayed in the `Documents` tab:

```
{
  "_id": ObjectId("6a32e939a453c779a23d8a9c"),
  "student": DBRef("students", "6a32e3f3f1dd87768b6ee4e"),
  "book": DBRef("books", "6a32e5fffd87768b6ee58"),
  "issueDate": "2026-06-17T18:30:00.000+00:00",
  "dueDate": "2026-07-01T18:30:00.000+00:00",
  "status": "RETURNED",
  "_class": "com.library.management.model.BorrowRecord"
}
```

5. Library cards(one to one with student)

The image displays two screenshots from a development environment. The top screenshot shows a REST client (likely Postman) with a POST request to `http://localhost:8081/api/library-cards`. The request body is a JSON array of two library card objects. The bottom screenshot shows the MongoDB Compass interface, displaying the results of the POST request in the `library_cards` collection of the `library_db` database. The results show two documents, each representing a library card with fields like `_id`, `cardNumber`, `student`, `status`, and `issueDate`.

REST Client Request:

```
POST http://localhost:8081/api/library-cards
{
  "cardNumber": "6a32e3f31dd877686beee4e",
  "issueDate": "2026-06-18",
  "status": "Active",
  "studentId": "6a32e3f31dd877686beee4e"
}
```

MongoDB Compass Results:

```
{
  "_id": "6a338316f0ff2166cc339892",
  "cardNumber": "6a32e3f31dd877686beee4e",
  "student": {
    "id": "6a32e3f31dd877686beee4e",
    "name": "Aditya Sharma Updated",
    "rollNo": "45",
    "email": "aditya.new@example.com",
    "department": {
      "id": "6a32df56c6853559d6c6d82",
      "name": null,
      "description": null
    }
  },
  "status": "Active",
  "issueDate": "2026-06-18T18:30:00.000+00:00"
}
```

```
{
  "_id": "6a32e58af1dd977686beee4f",
  "cardNumber": "123456789",
  "student": DBRef('students', '6a32e3f31dd877686beee4e'),
  "status": "Active",
  "issueDate": "2026-06-17T18:30:00.000+00:00"
}
```